

1. Manual Analysis with Banker's Algorithm (Initial State)

Given:

- Processes: P1, P2, P3, P4, P5
- Resource types: R1, R2, R3
- Total resources: (10, 5, 7)

Allocation matrix (from Table 1):

Process	R1	R2	R3
P1	7	5	4
P2	3	2	2
P3	9	0	2
P4	2	2	2
P5	4	4	4

Maximum demand matrix (same as Table 1):

Process	R1	R2	R3
P1	7	5	4
P2	3	2	2
P3	9	0	2
P4	2	2	2
P5	4	4	4

Available resources initially: (10, 5, 7)

Step 1: Compute Need = Max – Allocation

Process	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	0
P5	0	0	0

All Need vectors are zero because each process already holds its maximum demand. That means every process can finish immediately if it releases resources, but the system still needs to check if there is a safe sequence.

Step 2: Banker's Algorithm check

Since all processes have Need = (0,0,0), any process can finish without additional resources.

The available resources (10,5,7) are greater than or equal to zero for all.

We can pick any order. The state is **safe**.

Step 3: Safe sequence

One possible safe sequence: **P1 → P2 → P3 → P4 → P5** (or any permutation).

Reason: No process needs any extra resource, so they can all finish one by one.

2. Increasing Resources – R1 = 12 (Available: 12,5,7)

Available = (12,5,7).

Allocation and Max unchanged. Need remains all zeros.

State is still **safe** because available resources are even larger.

Safe sequence: any order (e.g., P1 → P2 → P3 → P4 → P5).

Explanation: More resources never hurt safety; the system remains safe.

3. Further Increase – R2 = 7 (Available: 12,7,7)

Available = (12,7,7).

Again, Need = (0,0,0) for all processes.

State is **safe**.

Safe sequence: any order.

Reflection on effect of additional resources:

- Adding more resources (when processes already hold their maximum) does not change the safe state – it remains safe.
 - In a real system, extra resources can reduce waiting time, prevent deadlocks, and improve throughput.
 - Here, because all needs are already satisfied, extra resources are simply idle but harmless.
-

4. C# Program – Banker's Algorithm Simulation

The program below prompts the user for:

- Number of processes & resource types
- Allocation matrix
- Maximum demand matrix
- Available resources

It then computes the **Need** matrix and runs the Banker's Algorithm to find a safe sequence.

If safe, it prints the sequence; otherwise, it reports unsafe.

Code

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
namespace BankersAlgorithm
```

```
{
```

```
    class Program
```

```
    {
```

```

static void Main(string[] args)
{
    Console.WriteLine("=== Banker's Algorithm Simulator ===\n");

    // Input number of processes
    int numProcesses = ReadInt("Enter number of processes: ");

    // Input number of resource types
    int numResources = ReadInt("Enter number of resource types: ");

    // Allocation matrix
    int[,] allocation = new int[numProcesses, numResources];

    Console.WriteLine("\nEnter Allocation matrix (each row: process, columns:
resources):");
    for (int i = 0; i < numProcesses; i++)
    {
        var values = ReadIntTokens($"P{i + 1}: ", numResources);
        for (int j = 0; j < numResources; j++)
            allocation[i, j] = values[j];
    }

    // Maximum demand matrix
    int[,] max = new int[numProcesses, numResources];

    Console.WriteLine("\nEnter Maximum demand matrix:");
    for (int i = 0; i < numProcesses; i++)
    {
        var values = ReadIntTokens($"P{i + 1}: ", numResources);
        for (int j = 0; j < numResources; j++)

```

```

        max[i, j] = values[j];
    }

    // Available resources
    int[] available = new int[numResources];

    Console.WriteLine("\nEnter Available resources (space separated):");

    var availInput = ReadIntTokens("> ", numResources);

    for (int j = 0; j < numResources; j++)
        available[j] = availInput[j];

    // Compute Need = Max - Allocation
    int[,] need = new int[numProcesses, numResources];

    for (int i = 0; i < numProcesses; i++)
        for (int j = 0; j < numResources; j++)
            need[i, j] = max[i, j] - allocation[i, j];

    // Banker's Algorithm
    bool[] finished = new bool[numProcesses];

    List<int> safeSequence = new List<int>();

    int[] work = (int[])available.Clone();

    bool progress;

    do
    {
        progress = false;

        for (int i = 0; i < numProcesses; i++)
        {
            if (!finished[i])

```

```

{
    bool canAllocate = true;
    for (int j = 0; j < numResources; j++)
    {
        if (need[i, j] > work[j])
        {
            canAllocate = false;
            break;
        }
    }

    if (canAllocate)
    {
        // Add resources back
        for (int j = 0; j < numResources; j++)
            work[j] += allocation[i, j];

        finished[i] = true;
        safeSequence.Add(i);
        progress = true;
    }
}

} while (progress);

// Check if all finished
bool isSafe = finished.All(f => f);
if (isSafe)
{

```

```

        Console.WriteLine("\nSystem is in a SAFE state.");

        Console.Write("Safe Sequence: ");

        for (int i = 0; i < safeSequence.Count; i++)
        {
            Console.Write($"P{safeSequence[i] + 1}");

            if (i < safeSequence.Count - 1) Console.Write(" -> ");

        }

        Console.WriteLine();
    }

    else
    {
        Console.WriteLine("\nSystem is in an UNSAFE state.");

        Console.WriteLine("No safe sequence exists.");
    }
}

// Reads a single integer with basic validation and reprompt
static int ReadInt(string prompt)
{
    while (true)
    {
        Console.Write(prompt);

        string? line = Console.ReadLine();

        if (line != null && int.TryParse(line.Trim(), out int value))
            return value;

        Console.WriteLine("Invalid input. Please enter an integer.");
    }
}

```

```

// Reads a line containing exactly expectedCount integers (space separated).
static int[] ReadIntTokens(string prompt, int expectedCount)
{
    while (true)
    {
        Console.Write(prompt);

        string? line = Console.ReadLine();

        if (line == null)
        {
            Console.WriteLine("Input ended unexpectedly.");
            Environment.Exit(1);
        }

        var tokens = line.Trim().Split((char[]?)null,
StringSplitOptions.RemoveEmptyEntries);

        if (tokens.Length != expectedCount)
        {
            Console.WriteLine($"Please enter exactly {expectedCount} values separated
by spaces.");
            continue;
        }

        var values = new int[expectedCount];

        bool ok = true;

        for (int i = 0; i < expectedCount; i++)
        {
            if (!int.TryParse(tokens[i], out values[i]))
            {

```



```

        Console.WriteLine("All entries must be integers. Please re-enter the line.");

        ok = false;

        break;
    }
}

if (ok) return values;
}
}
}
}

```

```

Microsoft Visual Studio Debug Console
=== Banker's Algorithm Simulator ===
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (each row: process, columns: resources):
P1: 7 5 4
P2: 3 2 2
P3: 9 0 2
P4: 2 2 2
P5: 4 4 4
Enter Maximum demand matrix:
P1: 7 5 4
P2: 3 2 2
P3: 9 0 2
P4: 2 2 2
P5: 4 4 4
Enter Available resources (space separated):
>
Please enter exactly 3 values separated by spaces.
> 9 5 7
System is in a SAFE state.
Safe Sequence: P1 -> P2 -> P3 -> P4 -> P5

```